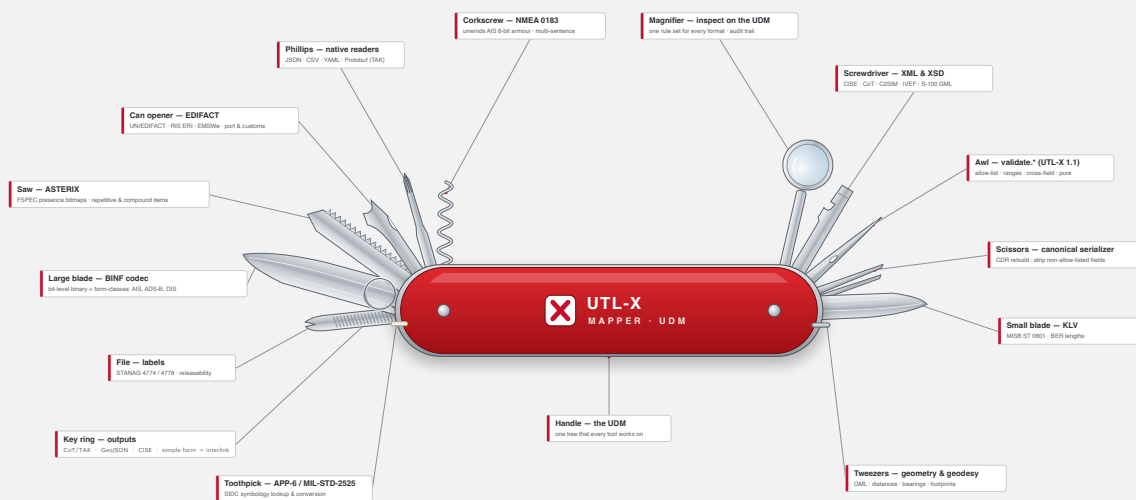


UTLX Mapper



UTL-X Guard — one engine, every open machine, addition and data format (Illustrative)

An assured message-mapping component
every open format, one model, validated on UTL-X 1.1

Ir. Marcel A. Grauwen

UTL-X — The Mapper

Copyright © 2026 Ir. Marcel A. Grauwen. All rights reserved.

Published by GLOMIDCO B.V., The Netherlands. Exploratory edition, 2026.

UTL-X is open-source software (AGPL-3.0), freely available at <https://github.com/grauwen/utl-x>. This book depends on **UTL-X 1.1** (the `validate.*` semantic-validation extension).

This is the short, component-focused companion to *UTL-X: One Language, All Formats* (the complete language specification) — for readers who want to understand UTL-X **as a mapping component** without reading the full language book. Its security counterpart is *UTL-X Guard*; its defence sibling is *UTL-X MIL — The Mapper* (restricted-format packs, higher assurance).

Status — exploratory. This book is a capability study of an assured, open-standards message-mapping component built on UTL-X. It is not a product specification. External standards are named indicatively; verify against primary sources.

Typeset with Typst in New Computer Modern.

Table of Contents

1	The UTL-X Mapper	7
1.1	What it is	7
1.2	At a glance	7
1.3	The Mapper and the Guard	7
1.4	What the rest of the book covers	8
2	Why Mapping Is the Bottleneck	9
2.1	The combinatorial problem	9
2.2	Two bridges that recur	9
2.3	What bespoke code costs	9
2.4	What a declarative mapper changes	9
2.5	Assured is validation, not just translation	10
3	What It Maps — the Format Landscape	11
3.1	Encoding decides the mechanism	11
3.2	The open formats	11
3.3	Open core	12
4	Deploying the Mapper Component	13
4.1	It is software, and it already runs in production shapes	13
4.2	Deployment forms	13
4.3	The sidecar model	13
4.4	Transport and buses	13
4.5	One mapping, every form — the Open-M model	14
4.6	Pipelines and chaining	14
4.7	Scaling and assurance	14
5	One Language, One Model	16
5.1	The shape of a mapping	16
5.2	The four guarantees	16
5.3	The core constructs	16
5.4	The Universal Data Model	17
5.5	Three accessors	17
6	Every Format Becomes One Tree	18
6.1	The problem with binary: it does not describe itself	18
6.2	Form-class: the layout as data	18
6.3	Why BINF, not “an ADS-B format”	18
6.4	The encoding spectrum, one codec	18
6.5	Mapping decisions: UDM does not come for free	19
6.6	Round-trip and re-encoding	19
7	Assured Validation	20
7.1	Why 1.0 is not enough	20
7.2	The <code>validate.*</code> namespace	20
7.3	ValidationResult: structured, auditable failure	21
7.4	It inherits the four guarantees	21
7.5	Validation as the contract gate	21
8	Message Contracts and the Mapping Workflow	22
8.1	What a message contract is	22
8.2	Schema-driven mapping	22
8.3	Instance and schema together	22
8.4	The authoring workflow	22
8.5	One rule, every format pair	23
9	The Standards in Depth	24

9.1	Binary — via a form-class	24
9.2	Text-structured — mapping only (no codec)	24
9.3	What the walk shows	25
10	Appendix A: UTL-X Quick Reference	26
10.1	Mapping skeleton	26
10.2	Accessors	26
10.3	Core constructs	26
10.4	UDM node kinds	26
10.5	validate.* (UTL-X 1.1)	26
10.6	The four guarantees	26
11	Appendix B: Format & Encoding Catalogue	27
11.1	Binary → BINF codec + form-class	27
11.2	XML family → native XML reader (mapping only)	27
11.3	JSON-alike → native reader (mapping only)	27
11.4	EDI / delimited → EDIFACT-class reader	27
11.5	Code systems → stdlib lookup (not a codec)	27
11.6	Out of core scope	27
11.7	What the engine already has vs. must build	27
12	Appendix C: Sources & Further Reading	28
12.1	Companion books	28
12.2	UTL-X design documents	28
12.3	Standards referenced (verify against primary sources)	28
12.4	A note on scope	28

PART I

The Mapper

What it is, the problem it solves, what it maps, and how it is deployed.

1 The UTL-X Mapper

A modern organisation runs on data that will not line up. One system speaks one format, the data platform another, a partner agency a third, and the cloud analytics stack a fourth. Between every pair sits a translator — and today that translator is almost always bespoke code, written once, understood by one team, and brittle the day a standard revises. The UTL-X Mapper is that translator, made declarative, auditable, and assured.

This is a short book, on purpose. If you want the full language, read *UTL-X: One Language, All Formats*. If you just want to understand UTL-X **as a mapping component** — what it does, what it maps, how it deploys, and why its output can be trusted — this is the on-ramp.

1.1 What it is

The Mapper is a **message-mapping component**: it takes a message in one format, turns it into a single internal model, and writes it out in another — correctly, repeatably, and provably. It reads the binary and the textual, the modern and the legacy, across every open standard, and maps between any of them with one language and one model.

parse any format → one model (UDM) → validate → emit any format

It is built on **UTL-X 1.1**. Version 1.0 gives the language its four guarantees — pure, stateless, deterministic, single-pass — which together make a mapping reproducible and testable. Version 1.1 adds the one thing a serious mapper cannot do without: **semantic validation** (`validate.*`), so the Mapper does not merely transform a message, it certifies that the result conforms to its contract before it is allowed out. That is the difference between a converter and an **assured** converter.

1.2 At a glance

What it does	maps a message from any supported format to any other, through one canonical model, with validation on the way out
Built on	UTL-X 1.1 — pure/deterministic transformation (1.0) plus semantic validation (<code>validate.*</code> , 1.1)
Formats	text and binary; open standards — JSON/XML/CSV/YAML/OData, EDIFACT, AIS, ADS-B, ASTERIX, MISB KLV, Cursor-on-Target, CISE and more
The model	one Universal Data Model (UDM): scalars, objects, arrays, binary, date-time, with data <code>.</code> , attribute <code>@</code> , and metadata <code>^</code> accessors
How it ships	as a software component — VM, container, Kubernetes workload, or a Dapr sidecar — embeddable in a pipeline or a service mesh
Connects via	any message bus or endpoint — Kafka, Pulsar, AMQP, JMS, MQTT, DDS, files — through the Dapr component model; fire-and-forget on a bus, or request/response as an API filter
What makes it assured	contract validation before emit; deterministic, reproducible, auditable output; open standards; and no AI in the mapping path

1.3 The Mapper and the Guard

This book has a companion, *UTL-X Guard*. The two share an engine but answer different questions. The Guard asks “**may this data cross?**” — it is a security device on a boundary, allow-list and fail-closed. The Mapper asks “**what does this data become?**” — it is an integration component in a pipeline, turning one system’s output into another’s input.

The Guard	security boundary · allow-list · fail-closed · inspects and blocks · “may it cross?”
------------------	--

The Mapper	integration pipeline · transform · validate · converts and delivers · “what does it become?”
-------------------	--

They are deliberately built from the same parts — the same UTL-X engine, the same UDM, the same binary codec, the same `validate.*`. A customer who fields the Guard already has the Mapper inside it; a customer who fields the Mapper can harden it into a Guard. This book is about the integration half of that story.

1.4 What the rest of the book covers

Part I is the product. Chapter 2 explains why mapping is the integration bottleneck and what a declarative mapper changes. Chapter 3 surveys what the Mapper reads and writes — the open-standards format landscape. Chapter 4 shows how the component is deployed.

Part II is the mechanism. Chapter 5 is the language and the model; Chapter 6 is how every format, including bit-level binary, becomes one tree; Chapter 7 is assured validation on UTL-X 1.1; Chapter 8 is message contracts and the authoring workflow; Chapter 9 walks the standards in depth. The appendices give a language quick reference, a format catalogue, and sources.

2 Why Mapping Is the Bottleneck

Integration is where data programmes slow down. Not because mapping is hard in principle — a field is a field — but because mapping is done the wrong way: as bespoke code, one connector at a time, each a small island of knowledge that decays the moment its author moves on. This chapter is the case for doing it differently.

2.1 The combinatorial problem

A real estate is not two systems; it is dozens. Sensors, applications, agency and partner feeds, logistics, and the modern IT and cloud estate behind them. Every pair that must exchange data needs a translation, and the number of possible pairs grows with the square of the systems. Built pairwise, that is an unbounded backlog of connectors — each written, tested, and maintained on its own.

A **canonical model** breaks the square. Map each format to one model once, and any format can reach any other through it: N readers and N writers instead of N^2 connectors. This is the oldest good idea in integration, and the UTL-X Mapper is built around it — the Universal Data Model of Chapter 5.

$$N^2 \text{ bespoke connectors} \rightarrow N \text{ readers} + N \text{ writers over one model}$$

2.2 Two bridges that recur

The combinatorial argument is abstract; the operational need is concrete. Two bridges recur:

Binary ↔ modern IT	an AIS position, an ADS-B squitter, an ASTERIX plot — born in a bit-packed binary format, needed as JSON or XML in a cloud analytics platform, a dashboard, or a data lake. The Mapper decodes the binary frame and emits the modern document, and back again.
New system ↔ existing picture	a newly fielded sensor, service, or partner system that must join an established data picture. Rather than modify the picture, write one mapping that makes the newcomer speak its format.

Both are the same operation — parse, model, validate, emit — and both are, today, usually a hand-written adapter nobody wants to own.

2.3 What bespoke code costs

Bespoke connector code carries four recurring costs, and each compounds over time:

Opacity	the mapping logic lives in a general-purpose language, legible only to its author; an integration or compliance reviewer cannot read the intent from the code
Fragility	a standard revises — a new ASTERIX category, a changed message field — and the change is a code change, a rebuild, and a redeployment, not a data edit
Inconsistency	the same conversion written twice by two teams differs in a dozen small ways; there is no single source of truth for “how we map a position”
Unassurability	arbitrary code can do anything — call the network, read a file, behave differently on the thousandth message; it cannot be certified as a pure, bounded transformation

2.4 What a declarative mapper changes

The UTL-X Mapper attacks all four by making the mapping a **declarative artefact** in a purpose-built language, not code in a general one:

Readable	a mapping reads as a description of the output in terms of the input; an integrator or a reviewer reads the intent directly (Chapter 5)
-----------------	---

Standards-as-data	a binary format is a definition file (a form-class, Chapter 6); a revised standard is an edited definition, not a rebuilt binary
One source of truth	one mapping per format pair, versioned and shared; “how we map a position” is a file, not a tribe
Assurable	the language is pure, deterministic, and single-pass — a mapping cannot reach the network or behave nondeterministically, so it can be tested exhaustively and certified (Chapter 7)

2.5 Assured is validation, not just translation

Best-effort conversion is often tolerated; a safety-, compliance-, or operations-critical feed is not. A malformed track, an out-of-range coordinate, a mislabelled or missing field — each is an operational error, and sometimes a compliance or security one. This is why the Mapper is built on UTL-X 1.1: every mapping can assert, in the same language, that its output conforms to the receiving contract — structure, value ranges, cross-field consistency, and labelling — before the message is allowed out (Chapter 7). The Mapper does not just translate; it refuses to emit a message it cannot certify.

With the **why** established, Chapter 3 shows the **what**: the landscape of open formats the Mapper reads and writes.

3 What It Maps — the Format Landscape

The Mapper’s reach is defined by the formats it reads and writes. This chapter surveys that landscape. The organising idea is deliberately counter-intuitive: formats are grouped not by **domain** — maritime, air, logistics — but by **encoding**, because the encoding, not the mission, decides what the Mapper has to do with a message.

3.1 Encoding decides the mechanism

A domain grouping tells you **who** uses a format. An encoding grouping tells you **what the engine must do** to read it. The single most important line runs between **binary** and **text-structured**:

Band	Self-describing?	Mapper mechanism
Bit-level binary	no — needs a form-class	BINF codec + form-class (Chapter 6)
Byte-structured binary (TLV/KLV)	partial	BINF, byte-granular mode
XML family (incl. GML)	yes — tags in the data	native XML reader (exists)
JSON-alike (JSON/YAML/Protobuf)	yes	native JSON/YAML/Protobuf (exists)
EDI / delimited (EDIFACT)	yes, via grammar	EDIFACT-class reader
Code systems / vocabularies	n/a	stdlib lookup, not a codec

The practical consequence: the whole text-structured half — every XML and JSON format — is **mapping only**, because UTL-X already reads and writes it. The new engineering is concentrated in one place, **BINF** (the binary half, Chapter 6), with a separate EDIFACT reader for the delimited dialects. That is what makes the breadth below affordable: most of it is configuration, not code.

3.2 The open formats

Everything the Mapper reads is publicly specified and buildable with no restrictions.

Format	Encoding	Role
AIS	6-bit-packed binary in NMEA armour	maritime positions; the ideal first binary pack
ADS-B (1090ES)	fixed 112-bit frames + 24-bit CRC	air positions; type-code discriminator, CPR position
ASTERIX	binary, FSPEC presence bitmaps	radar / surveillance; the variable-structure stress test
MISB KLV ST 0601	KLV byte-structured binary	full-motion-video metadata; self-tagged TLV
DIS	binary PDUs, fixed layouts	simulation record / replay
Cursor-on-Target (CoT)	XML (TAK also Protobuf)	the canonical map-to target for decoded tracks
CISE	XML	EU maritime surveillance data model — a flagship civilian bridge
C2SIM / IVEF	XML	command-and-control ↔ simulation; VTS-centre exchange
IHO S-100 (GML part)	GML (= XML)	hydrographic; GML handled as XML

UN/EDIFACT, RIS	segment/element delimited text	ports, customs, logistics, inland shipping — huge installed base
APP-6 / MIL-STD-2525	symbol identity codes (SIDC)	a public symbology lookup/enum, e.g. identity → CoT type

The natural first bridge — and the Mapper’s “hello world” — is **AIS** → **CoT**: decode a bit-packed maritime position and emit it as a Cursor-on-Target event, ready for a TAK client. It exercises the binary codec on the way in and a native reader on the way out, end to end.

3.3 Open core

The crucial design line: the **engine is content-free**. UTL-X itself ships no domain definitions. A binary format is a **definition pack** — a form-class file (Chapter 6) — and the publicly specified ones (AIS, ADS-B, ASTERIX, KLV, CoT, CISE...) are open. The engine, the UDM, the BINF codec and the native readers are open source; the commercial layer is productisation — contract and policy tooling, support, packaging — not the engine.

Open-core engine	the UTL-X 1.1 engine, UDM, BINF codec, native readers — no embedded content
Open packs	AIS, ADS-B, ASTERIX, KLV, CoT, CISE, EDIFACT... — publicly buildable definitions

Specialised, access-restricted standards (the defence tactical data links) are not part of this edition; they are a separately governed edition, supplied by their holder under their own access rights, and never embedded in the open engine.

With the landscape mapped, Chapter 4 turns to how the component is deployed into a pipeline — and Chapter 9 returns to these standards in depth.

4 Deploying the Mapper Component

A mapper is useful only where the data flows. The UTL-X Mapper is a **software component**, so it drops into the places integration actually happens — a pipeline stage, a message-bus subscriber, a service-mesh sidecar, an API filter. This chapter covers the forms it takes and the model that lets one mapping run unchanged across all of them.

4.1 It is software, and it already runs in production shapes

The Mapper is not new infrastructure. UTL-X already runs as a GraalVM-native binary and as a managed cloud component — the *UTLXe on Azure* guide runs the engine as a container with **Dapr** pub/sub and bindings. The Mapper is that engine, configured with the format packs and the `validate.*` contracts of this book. Everything below is a deployment **form** of one component, not a separate product per form.

4.2 Deployment forms

VM image (OVA)	for traditional, virtualised, and air-gapped estates; the whole engine in one appliance image
OCI container / Kubernetes	one pod per flow, managed by an operator; packs and contracts distributed as signed artefacts; fits the Open-M mode: component model
Dapr sidecar / middleware	a mapping step beside a workload, on pub/sub or input/output bindings — every service gets format translation with no code change
API-gateway filter	the Mapper as a request/response transform in front of an API — inbound and outbound shaping
Embedded library / CLI	the native binary invoked in a batch pipeline, a build step, or a ground-station script

4.3 The sidecar model

The most cloud-native form is the Mapper as a **sidecar**: it sits beside a service and translates on the way in and out, so a component that speaks only JSON can transparently consume AIS, ADS-B, or a binary feed.

```
binary feed → [ UTL-X Mapper sidecar ] → app (speaks JSON/XML)
                parse → UDM → validate → emit
                invalid → dead-letter + audit (never a bad message in)
```

Via Dapr bindings or pub/sub middleware, the mapping is injected around the service without touching its code. The honest boundary: a sidecar **shares the pod's trust domain**, so this is the right shape for **integration and data-shaping inside one trust zone** — not for a cross-domain security crossing, which is the Guard's job and which insists that each side run its own box.

4.4 Transport and buses

A mapping decides on **content**, not on the wire a message arrived on, so the transport is an **adapter layer** around it: which bus or endpoint carries messages in and out is a configuration choice, not a change to the mapping. Reusing UTL-X's Dapr component model (pub/sub and input/output bindings), the Mapper connects to a broad set of transports through configuration rather than code:

Message buses / brokers	Apache Kafka, Apache Pulsar, AMQP (RabbitMQ), MQTT, JMS-style brokers, and managed cloud queues (Azure Service Bus, AWS SQS/SNS, Google Pub/Sub)
--------------------------------	--

Real-time pub-sub	OMG DDS — the SOSA/FACE-aligned real-time data bus — and specialised feeds once they reach IP
Direct / batch	TCP/UDP, HTTP/REST and gRPC endpoints, file and directory drops, serial — for sensors, ground stations and batch transfer

The same mapping serves both delivery styles — which is where the Mapper differs from the Guard. As a **bus subscriber or pipeline stage** it is a **fire-and-forget forward**: consume a message, map it, and publish the result downstream (invalid input → dead-letter + audit, never a bad message onward). As an **API-gateway filter** it runs **request/response**, shaping a call inbound and its reply outbound. The Guard deliberately permits only the one-way, fire-and-forget form — a reply path across a trust boundary would be a back-channel — whereas the Mapper, an integration component inside one trust zone, is free to do either.

4.5 One mapping, every form — the Open-M model

The property that makes the component economics work: a mapping is written **once** and runs **unchanged** across every form above. The deployment is a packaging and operations decision, not a rewrite — the same leverage Solace gets by shipping one messaging product as both an appliance and a VM/container. Under the Open-M **mode: component** model a mapping is declared with its input and output formats and dropped into whichever runtime the deployment needs:

```
%utlx 1.1
input  ais
output cot
---
// the mapping body – identical whether this runs as a
// container, a Dapr sidecar, an API filter, or the CLI
```

4.6 Pipelines and chaining

Because every mapping is a pure function from one format to another, mappings **compose**. A binary feed can be decoded, enriched from a lookup pack, validated against a receiving contract, and re-emitted in a modern format as a chain of component stages — each independently testable, each auditable:

```
AIS frame → [decode AIS] → [enrich: MMSI → vessel] → [validate CISE contract]
→ CISE XML
```

Each stage is one UTL-X mapping; the chain is a pipeline of components. A standards revision changes one stage's definition, not the pipeline.

4.7 Scaling and assurance

As a pure, stateless, single-pass function (Chapter 5), each mapping instance is independent — so throughput scales **horizontally** by running more instances behind a load balancer or more pods in a deployment, with no shared state to coordinate. Where assurance rather than throughput is the driver, the same component climbs a hardening ladder — hardened container, dedicated VM, confidential-computing enclave — exactly as the companion Guard does, without changing the mapping. That assurance story, and the hardware that backs the highest tiers, is the subject of the Guard book; the Mapper inherits it unchanged.

Part II now opens the component up — starting with the language and the model that make a mapping a pure, composable function in the first place.

PART II

How It Works

One language, one model, binary decoding, validation, and contracts.

5 One Language, One Model

Everything the Mapper does rests on two foundations: a small declarative language, and one internal model every format flows through. This chapter introduces both — enough to read every mapping in the book.

5.1 The shape of a mapping

A UTL-X mapping is a document with two parts: a **header** declaring the version and the input/output formats, and a **body** describing the output as an expression over the input.

```
%utlx 1.1
input  ais
output cot
---
{
  event: {
    "@uid":    $input.mmsi,
    "@type":   "a-f-S",          // a surface track
    point: { "@lat": $input.lat, "@lon": $input.lon }
  }
}
```

The header's `%utlx 1.1` fixes the language version; `input/output` name the formats (resolved to readers and writers, including BINP packs — Chapter 6). The body is a single expression: no statements, no mutation, no control flow in the imperative sense — you write **what the output is**, not **how to build it step by step**.

5.2 The four guarantees

UTL-X 1.0 holds a mapping to four properties, and the Mapper's assurance story depends on all four:

Pure	a mapping has no side effects — no file, no network, no clock; output depends only on input
Stateless	no memory between messages; the thousandth message maps exactly as the first
Deterministic	the same input always produces the same output — so a mapping can be tested exhaustively and its result reproduced in audit
Single-pass	a mapping reads its input once, in one pass — bounded time and memory, suitable for line-rate and for a hardware fast-path

Together these make a mapping a **mathematical function**: safe to run inline on a connection, safe to certify, and impossible to turn into a covert channel. UTL-X 1.1 adds validation (Chapter 7) without breaking any of them.

5.3 The core constructs

The body language is small. These are the constructs that appear throughout the book:

let	bind a name to a sub-expression for reuse and readability
map / filter	transform and select over arrays — the workhorses of record-to-record mapping
 >	the pipe: feed a value through a sequence of transformations left to right
match	branch on shape or value — e.g. dispatch on an ADS-B type code or an ASTERIX category
??	the default operator: supply a fallback when a value is absent

reduce	fold an array to a single value — totals, concatenations, aggregates
---------------	--

```
%utlx 1.1
input  asterix
output json
---
{
  tracks: $input.records
    |> filter(r => r.category == 48)          // radar tracks only
    |> map(r => {
      id:  r.trackNumber,
      alt: r.flightLevel ?? 0,              // default when absent
      kind: match r.type {
        0 => "unknown",
        1 => "primary",
        2 => "secondary"
      }
    })
}
```

5.4 The Universal Data Model

Every format, once read, becomes a **UDM tree** — one model with a handful of node kinds. The mapping body never touches JSON or bits or EDI segments; it only ever navigates a UDM tree. That is the whole trick behind “every format, one rule set.”

Scalar	a string, number, boolean, or null — a leaf value
Object	an ordered set of named children
Array	an ordered sequence of nodes
Binary	a raw byte payload (e.g. an opaque sub-field) carried without interpretation
DateTime	a first-class instant, so date logic is uniform across formats

5.5 Three accessors

UDM nodes carry three kinds of information, each with its own accessor — a distinction that matters enormously for sensitive data, where **about-the-data** (the provenance, the source) is as important as the data:

. — data	the value itself: <code>\$input.lat</code> , <code>track.id</code>
@ — attribute	structural attributes native to the format: an XML attribute, a CoT <code>@type</code>
^ — metadata	out-of-band facts about a node: provenance, confidence, and the security label that drives releasability (Chapter 7)

The **^** metadata channel is how a classification caveat, a source sensor, or a confidence score rides alongside a value through the whole mapping — present for validation and policy, never silently promoted into the data. With the language and the model in hand, Chapter 6 shows how even a bit-packed binary frame becomes one of these trees.

6 Every Format Becomes One Tree

The UDM of Chapter 5 is only useful if every format can reach it. For text-structured formats that is easy — UTL-X already reads XML and JSON. The hard, and interesting, case is **binary**: an AIS burst, an ADS-B squitter, an ASTERIX record — bits packed with no delimiters and no tags. This chapter is how those become trees too.

6.1 The problem with binary: it does not describe itself

A JSON document carries its own structure — braces, keys, values. A bit-packed frame does not. An ADS-B 1090ES message is 112 bits with a 24-bit CRC; nothing **in** those bits says “the next 5 bits are a type code.” The layout lives **outside** the data, in the specification. To read the frame, the engine needs that layout supplied to it.

6.2 Form-class: the layout as data

The oldest name for a non-self-describing record whose shape is defined externally comes from the trading floor: a **form-class**. UTL-X adopts the term. A form-class is a **definition** — a declarative description of a binary format’s physical layout: the fields, their bit offsets and widths, their encodings, scalings, enumerations, and discriminators. It is **data**, not code.

```
%utlx 1.1
input  binf { definition: "packs/ads-b.def" }
output json
---
{ icao: $input.icao, altitude: $input.alt, callsign: $input.callsign }
```

The **BINF** codec reads the raw bytes, consults the form-class, and produces a UDM tree — the same kind of tree a JSON reader produces. From that point on the mapping is format-agnostic: it navigates a tree, indifferent to the fact that the tree came from bits.

raw bytes + form-class (definition) → BINF codec → UDM tree

6.3 Why BINF, not “an ADS-B format”

A fair question: why declare `input binf { definition: "ads-b.def" }` rather than simply `input ads-b`? Because the generic mechanism is the asset. ADS-B is **one** fixed-112-bit format; the integration world has dozens of bit-packed formats, and new ones arrive with each standards revision. Baking each into the engine as a named reader would mean a code change per format and a content-bearing engine. Making the engine a **generic bit-level codec** driven by a form-class means a new format is a new **definition file** — buildable by whoever holds the spec, with no engine change and nothing domain-specific baked into the engine.

Named aliases are then a convenience on top: an operator can bind `input ads-b` to `binf { definition: "packs/ads-b.def" }` so day-to-day mappings read cleanly, while the engine stays generic underneath.

6.4 The encoding spectrum, one codec

BINF spans the binary bands of Chapter 3 with one mechanism:

Bit-level	fields at arbitrary bit offsets, no delimiters — AIS, ADS-B
Byte-structured	records/PDUs on byte boundaries — DIS PDUs
Self-tagged (TLV/KLV)	key-length-value, partly self-describing — MISB KLV ST 0601
Presence-driven	existence bitmaps declaring which optional fields are present — ASTERIX FSPEC

The presence-driven case is the demanding one, and it recurs across several binary formats — which is why ASTERIX is the proving ground: get FSPEC right and the harder presence-driven form-classes follow the same pattern.

6.5 Mapping decisions: UDM does not come for free

Does a bit-field map cleanly onto a UDM scalar, object, or array? Often, but not always — and pretending otherwise would be dishonest. A form-class must make real decisions, and they are design choices, not mechanical ones:

Scaling	a raw integer that means “latitude $\times 10^7$ ” becomes a UDM number — the scaling lives in the definition
Enumerations	a 3-bit code becomes a meaningful string (or stays numeric) — a definition choice
Sentinels	a reserved “value not available” encoding must become <code>null</code> , not a misleading number
Multi-frame state	ADS-B CPR position needs two frames to resolve a position — a bounded, declared exception to pure per-message decoding
Opaque sub-fields	a field BINF should not interpret is carried as a UDM Binary node, untouched

These decisions are exactly where format expertise lives, and exactly why a **declarative** definition is the right home for them: they are reviewable, versionable, and changed as data when the standard revises — not buried in a decoder’s source.

6.6 Round-trip and re-encoding

Because a form-class describes a layout, BINF reads **and writes** it: a mapping can emit a bit-packed frame as readily as it consumes one, re-encoding from the UDM tree. That is what makes true binary $\leftrightarrow$ binary and binary $\leftrightarrow$ modern bridges possible — decode an AIS position to the UDM, and re-encode it as CoT XML, or as JSON, from the same model. Because every format meets in one model, re-encoding is tractable rather than lossy.

With every format now a tree, Chapter 7 adds the step that makes the Mapper **high-assurance**: asserting that the tree is correct before it is allowed out.

7 Assured Validation

A converter translates. An **assured** converter refuses to emit a message it cannot certify as correct. That refusal is the whole point of building the Mapper on UTL-X **1.1** rather than 1.0 — and this chapter is how it works.

7.1 Why 1.0 is not enough

UTL-X 1.0 is a transformation language: it maps, converts, restructures. It does **not** validate. If a field is wrong, missing, or internally inconsistent, 1.0 maps it anyway — it has no first-class way to assert a value is correct before proceeding. The workarounds (a boolean flag routed downstream, null sentinels, a hand-written Kotlin component) all lose error detail or push the logic out of the declarative mapping where it belongs. Cross-field correctness — that a coordinate is in range, that a track’s fields are mutually consistent, that a release caveat is a permitted value — is a natural part of an assured mapping. It belongs **in the language**.

7.2 The `validate.*` namespace

UTL-X 1.1 adds one thing: the `validate.*` standard-library namespace. Each function takes the current value and returns a `ValidationResult` — a first-class UDM node recording what passed, what failed, and why.

<code>validate.assert(rules)</code>	the general form: a list of named rules, each a condition + message, with an optional field path and severity (<code>error</code> / <code>warning</code>)
<code>validate.required(fields)</code>	assert that named fields are present
<code>validate.inRange(field, min, max)</code>	assert a numeric field lies within bounds
<code>validate.oneOf(field, values)</code>	assert a field is one of an allowed set — the releasability/enum check
<code>validate.matches(field, pattern)</code>	assert a field matches a regular expression
<code>validate.isoDate(field, format)</code>	assert a date-time field is well-formed
<code>validate.unique(arrayField, keyField)</code>	assert no duplicate keys in a collection
<code>validate.mutuallyExclusive(fields)</code>	assert at most one of a set is present
<code>validate.atLeastOne(fields)</code>	assert at least one of a set is present

They chain with the pipe, and the results merge into one `ValidationResult`:

```
%utlx 1.1
input ais
output cise
---
let checked = $input
  |> validate.required(["mmsi", "lat", "lon"])
  |> validate.inRange("lat", -90.0, 90.0)
  |> validate.inRange("lon", -180.0, 180.0)
  |> validate.oneOf("navStatus", [0,1,2,3,4,5,6,7,8,9,10,11,14,15])

checked.valid
? { vessel: { id: checked.payload.mmsi, position: {...} } }
: dead_letter(checked.errors)
```

7.3 ValidationResult: structured, auditable failure

The `ValidationResult` node is what turns validation from a yes/no into evidence:

<code>valid</code>	true if no error -severity rule failed (warnings do not clear it)
<code>passed / failed / warnings</code>	counts, for dashboards and metrics
<code>errors[]</code>	per-failure detail: the rule id, the human message, the field path(s), and the severity
<code>payload</code>	the value under validation — passed through, so downstream expressions see the data directly

Every rejected message therefore carries **why** it was rejected — which rule, which field — as a structured record, not a log line. For an assured mapper this is decisive: a rejection is auditable evidence, and an operator or an accreditor can see exactly which contract a feed violated.

7.4 It inherits the four guarantees

Crucially, `validate.*` breaks **none** of the 1.0 guarantees. Every function is pure (same input → same `ValidationResult`), stateless (no lookup, no I/O, no model weights), and deterministic (no randomness, no probability). This is exactly what makes 1.1 a **minor** version and not a major one: it only adds. The contrast is deliberate — the `ai.*` namespace proposed for UTL-X 2.0 **would** break purity and determinism, which is why it needs a major version and why **AI never enters the mapping path**. An assured verdict must be reproducible; a probabilistic one is not.

Guarantee	1.0 transform	1.1 validate
Pure	✓	✓
Stateless	✓	✓
Deterministic	✓	✓
Single-pass	✓	✓

7.5 Validation as the contract gate

The operational pattern the Mapper is built around: **validate against the receiving contract before emit**. A mapping does not merely produce output in the target format — it asserts the output conforms to what the receiver will accept (its required fields, value ranges, code lists, releasability labels), and routes a failure to a dead-letter with its `ValidationResult` instead of delivering a bad message into a downstream system.

parse → map to UDM → validate against the contract → emit only if valid, else dead-letter + audit

The releasability check deserves emphasis: a STANAG 4774/4778 confidentiality label rides on the ^ metadata channel (Chapter 5), and `validate.oneOf` on that label is how the Mapper enforces that a message carries a permitted caveat for its destination — mislabelled data is refused, not translated.

Chapter 8 turns this gate into a workflow: how contracts are defined and how a mapping is authored and proven against them.

8 Message Contracts and the Mapping Workflow

A mapping does not exist in isolation; it exists between two **contracts** — what the sender produces and what the receiver accepts. This chapter is about making those contracts explicit, and about the workflow of authoring, validating, and proving a mapping against them.

8.1 What a message contract is

A message contract is the agreed shape of a message at a boundary: its format, its schema (which fields, of which types), its value constraints, and its labelling rules. Integration goes wrong when contracts are **implicit** — living in a sender’s head and a receiver’s code — and it goes right when they are **explicit artefacts** a mapping can be written and tested against.

The Mapper treats both ends as first-class:

Input contract	the format and schema the Mapper reads — a form-class for binary (Chapter 6), or an XML/JSON schema for text
Output contract	the format, schema, value ranges, code lists, and labelling the receiver will accept — enforced by <code>validate.*</code> before emit (Chapter 7)

8.2 Schema-driven mapping

When both contracts are declared as schemas, authoring a mapping becomes a guided, checkable task rather than guesswork. The Mapper’s authoring environment works from the two schemas directly:

Field discovery	both contracts present as navigable trees — the input UDM shape and the output’s required shape side by side
Coverage	every required output field is tracked; an unmapped required field is flagged before the mapping ships, not after a message is rejected
Type awareness	the mapping knows a field is a scaled number, an enum, or a date, and the matching <code>validate.*</code> check is suggested
Label mapping	the ^ metadata channel is mapped explicitly, so a classification or releasability caveat is never dropped or silently invented

8.3 Instance and schema together

A contract has two faces: the **schema** (the rule) and an **instance** (an example message). The Mapper’s workflow uses both — the schema to check coverage and types, a real instance to see the mapping run end to end. A binary input instance decodes through its form-class to a UDM tree; the mapping produces an output instance; the output is validated against the output contract. The author sees, on one screen, the input, the result, and the verdict.

A subtlety worth stating, because it is easy to get wrong: a **schema** format and its **instance** format are not the same thing. A JSON-schema contract describes an instance that is JSON — or YAML, which shares the model. The authoring environment keeps that mapping explicit (a JSON schema admits JSON and YAML instances; an XML schema admits XML) so the example a mapping is tested against is always a **valid instance of the declared contract**, never a schema document mistaken for data.

8.4 The authoring workflow

Authoring a Mapper mapping follows one loop:

1. Declare contracts	name input and output formats; attach the form-class / schema for each
2. Map	write the body — navigate the input tree, build the output tree; coverage tracks the required fields

3. Validate	add the <code>validate.*</code> gate for the output contract — required fields, ranges, code lists, labels
4. Prove	run real instances through the mapping; confirm valid messages pass and malformed ones land in dead-letter with a <code>ValidationResult</code>
5. Ship	the mapping is a versioned artefact; a standards revision re-enters the loop at step 1 as a definition/schema edit

Because a mapping is pure and deterministic (Chapter 5), step 4 is **exhaustive and repeatable**: a corpus of instances — valid, boundary, and deliberately malformed — forms a regression suite that proves the mapping’s behaviour and re-proves it after every change. That test corpus is itself an accreditation asset.

8.5 One rule, every format pair

The payoff of the canonical model returns here. A validation rule — “latitude in $[-90, 90]$ ”, “release caveat in the permitted set” — is written **once** against the UDM and applies no matter which format the data arrived in or which it is leaving as. The contract library is not per-format-pair; it is a set of reusable checks over one model, composed per output contract. That is what keeps a growing matrix of format pairs maintainable instead of combinatorial.

With the workflow established, Chapter 9 grounds all of it in the actual standards — walking the open formats the Mapper bridges, and what each demands of a form-class and a contract.

9 The Standards in Depth

This chapter walks the open formats the Mapper bridges — what each one is, how it is encoded, and what it asks of a form-class and a contract. It is organised the way the engine sees them: the **binary** formats that need a form-class, and the **text-structured** formats that do not.

9.1 Binary — via a form-class

9.1.1 AIS — Automatic Identification System

Maritime self-reporting: position, identity, voyage. Encoded as **6-bit-packed binary inside NMEA armour**, with scaling, enumerations, and reserved sentinels. It is the Mapper’s ideal first binary pack — small, complete, and exercising every form-class feature that matters (scaling, enums, “not available” sentinels). The canonical demonstration is **AIS → CoT**.

9.1.2 ADS-B — 1090ES Extended Squitter

Air self-reporting. Fixed **112-bit Mode S frames with a 24-bit CRC**, a type-code discriminator, and **CPR** position encoding that needs two frames to resolve a location — a bounded, declared multi-frame exception to pure per-message decoding (Chapter 6). A fixed-layout form-class with a **match** on the type code.

9.1.3 ASTERIX — EUROCONTROL surveillance

The radar / surveillance exchange. Binary records whose present fields are declared by **FSPEC** presence bitmaps, organised into **categories** (e.g. CAT021 for ADS-B, CAT048 for radar). It is the Mapper’s variable-structure stress test — the demanding presence-driven pattern, on a public standard.

9.1.4 MISB KLV ST 0601 — full-motion-video metadata

FMV metadata as **KLV** (key-length-value) — self-tagged, byte-structured binary. Because it is self-describing via keys, it is a clean byte-granular BINF demonstration, and fully public.

9.1.5 DIS — Distributed Interactive Simulation

Binary PDUs with fixed layouts, for simulation record and replay. A straightforward byte-structured form-class, useful for training and test harnesses.

9.2 Text-structured — mapping only (no codec)

9.2.1 Cursor-on-Target (CoT) and CISE

The two canonical **map-to** targets. CoT is a compact **XML** event (position + detail), the lingua franca of TAK clients — the destination for a decoded AIS or ADS-B track. CISE is the EU’s **XML** maritime surveillance data model — the flagship civilian bridge: CISE requires an adaptor per connected system, and the Mapper is an adaptor with validation. Both are native-XML, mapping-only.

9.2.2 C2SIM, IALA IVEF, IHO S-100

Command-and-control ↔ simulation (C2SIM), VTS-centre exchange (IVEF), and hydrographic products (IHO S-100, GML part) — all **XML** families the native reader already handles; the work is **mapping**, not decoding.

9.2.3 EDIFACT and RIS

UN/EDIFACT is **segment / element delimited text** — the ports, customs and logistics backbone, with an enormous installed base; it needs the dedicated EDIFACT-class reader (not BINF, not XML). RIS (River Information Services — ERI, NtS, Inland AIS) is an umbrella that routes to AIS, EDIFACT and XML sub-formats by part.

9.2.4 APP-6 / MIL-STD-2525 symbology

Not a message format but a **public symbol-identity code system** (SIDC) — a lookup used inside a mapping to turn an identity into, say, a CoT type. Handled as a stdlib lookup, not a codec.

9.3 What the walk shows

Three things recur across every entry, and they are the thesis of the book in miniature. First, the **encoding**, not the domain, decides the work — binary needs a form-class, XML does not. Second, the hard binary patterns are **shared** — presence bitmaps, fixed words, discriminators — so getting one right (ASTERIX's FSPEC) carries over to the next. Third, every format is **data** to the engine — a definition pack or a mapping — so breadth is configuration, not code, and the open-core engine stays content-free.

10 Appendix A: UTL-X Quick Reference

A one-page reference for reading and writing the mappings in this book. For the complete specification, see UTL-X: One Language, All Formats.

10.1 Mapping skeleton

```
%utlx 1.1
input <format>           // e.g. json, xml, ais, binf { definition: "packs/x.def" }
output <format>
---
<expression>           // the output, as one expression over $input
```

10.2 Accessors

.	data — \$input.lat, track.id
@	attribute — event."@type", an XML attribute
^	metadata — provenance, confidence, security label (releasability)

10.3 Core constructs

let name = expr	bind a reusable sub-expression
map(x => ...)	transform each element of an array
filter(x => cond)	keep elements matching a condition
reduce(f, init)	fold an array to one value
a > f > g	pipe: feed a value through transforms left to right
match v { p => e, ... }	branch on value or shape
a ?? b	default: b when a is absent/null

10.4 UDM node kinds

Scalar · Object · Array · Binary · DateTime

10.5 validate.* (UTL-X 1.1)

validate.assert(rules)	named rules: condition + message (+ field, severity)
validate.required(fields)	fields must be present
validate.inRange(field, min, max)	numeric bound
validate.oneOf(field, values)	field ∈ allowed set
validate.matches(field, pattern)	regex match
validate.isoDate(field, format)	well-formed date-time
validate.unique(arrayField, keyField)	no duplicate keys
validate.mutuallyExclusive(fields)	at most one present
validate.atLeastOne(fields)	at least one present

Chain with |>; results merge into one `ValidationResult { valid, passed, failed, warnings, errors[], payload }`. Downstream expressions see `payload` directly.

10.6 The four guarantees

pure · stateless · deterministic · single-pass — held by both 1.0 (transform) and 1.1 (validate). `ai.*` (2.0) is excluded from the mapping path because it breaks purity and determinism.

11 Appendix B: Format & Encoding Catalogue

Every format in the civilian scope, by encoding band, with the Mapper mechanism it needs. All are publicly specified. “Mechanism” is what the engine does to read it.

11.1 Binary → BINF codec + form-class

AIS	6-bit-packed binary (NMEA armour)
ADS-B (1090ES)	fixed 112-bit frames + 24-bit CRC
ASTERIX	binary, FSPEC presence bitmaps
MISB KLV ST 0601	KLV (byte-structured, self-tagged)
DIS	binary PDUs, fixed layouts

11.2 XML family → native XML reader (mapping only)

Cursor-on-Target (CoT)	XML (TAK also Protobuf)
CISE	XML
C2SIM	XML (ontology-based)
IALA IVEF	XML
IHO S-100 (GML part)	GML (= XML)

11.3 JSON-alike → native reader (mapping only)

JSON / YAML / OData	structured text — the modern-IT side of a bridge
Protobuf (incl. TAK)	byte-aligned, schema-driven

11.4 EDI / delimited → EDIFACT-class reader

UN/EDIFACT	segment / element delimited
EMSWe	XML + EDIFACT (XML → native, EDI → EDIFACT reader)
RIS	umbrella: AIS + EDIFACT + XML (routes by sub-format)

11.5 Code systems → stdlib lookup (not a codec)

APP-6 / MIL-STD-2525	symbol identity codes (SIDC) — a public lookup/enum used inside a mapping
----------------------	---

11.6 Out of core scope

IHO S-100 (ISO 8211 / HDF5)	complex random-access containers — library-backed, not a form-class
HLA (RTI)	a simulation runtime API, not a wire format — bridge DIS instead

11.7 What the engine already has vs. must build

Capability	Status	Serves
XML reader/writer	exists	CoT, CISE, C2SIM, IVEF, GML
JSON / YAML / Protobuf / OData	exists	modern-IT bridge, TAK Protobuf
BINF codec + form-class	new	all binary (AIS, ADS-B, ASTERIX, KLV, DIS)
EDIFACT-class reader	new	EDIFACT, EMSWe-EDI, RIS-EDI
Code-set lookup (stdlib)	partial	APP-6 / 2525

12 Appendix C: Sources & Further Reading

The design documents, specifications, and companion books behind this edition. Standards references are indicative; verify each against its primary source.

12.1 Companion books

<i>UTL-X Guard</i>	the security counterpart: the same engine as a content guard — allow-list, fail-closed, the verdict policy, deployment shapes
<i>UTL-X: One Language, All Formats</i>	the complete UTL-X language specification
<i>UTLXe on Azure — Deployment and Operations Guide</i>	running the engine as a managed cloud component with Dapr

12.2 UTL-X design documents

utl-x/docs/proposals/utlx-1_1-semantic-validation.md	the UTL-X 1.1 <code>validate.*</code> namespace and <code>ValidationResult</code> — the validation this book depends on
utl-x/docs/proposals/utlx-language-versioning-validation.md	versioning and packaging: why <code>validate.*</code> is 1.1 and <code>ai.*</code> is 2.0; UTLXe vs UTLXS
utl-x/docs/proposals/utlx-validate-vocabulary-coverage.md	the <code>validate.*</code> vocabulary, audited against Schematron and FEEL
BINF-bit-level-binary-format.md	the BINF codec and form-class: binary decoding, named aliases, ^ metadata, UDM mapping decisions
formats-open/README.md	the encoding-grouped classification of the open formats and the proving order

12.3 Standards referenced (verify against primary sources)

Surveillance & tracks	EUROCONTROL ASTERIX; ITU-R M.1371 (AIS); RTCA DO-260B (ADS-B 1090ES); MISB ST 0601 (KLV)
Overlays & symbology	Cursor-on-Target; APP-6 / MIL-STD-2525 symbology (public); C2SIM
Maritime & logistics	EU CISE; IALA IVEF; IHO S-100; UN/EDIFACT; EMSWe; RIS (ERI, NtS, Inland AIS)
Labelling	STANAG 4774 (confidentiality label syntax) / 4778 (binding) — public; referenced for label validation

12.4 A note on scope

Every format in this book is publicly specified and buildable without restriction — no sponsor, no clearance, no restricted access. The engine stays content-free: a binary format is a **definition pack**, and the ones here are open. Specialised, access-restricted standards (the defence tactical data links) are a separately governed edition, not part of this book.

UTL-X — The Mapper · Exploratory edition, 2026 · An assured message-mapping component built on UTL-X 1.1.